

СЪДЪРЖАНИЕ

1.	Увод	3
2.	АНАЛИЗ НА ПРОБЛЕМА И СЪЩЕСТВУВАЩИТЕ РЕШЕНИЯ.....	5
2.1	Дефиниране и същност на проблема	5
2.2	Анализ на архитектурата „Клиент-Сървър“ в контекста на проекта.....	6
2.3	Преглед на съществуващите решения и техните ограничения	7
2.4	Формулиране на изискванията към софтуерната система.....	8
3.	ОПИСАНИЕ И АРГУМЕНТАЦИЯ НА ИЗБОРА НА ТЕХНОЛОГИИ.....	11
3.1	Въведение в технологичния стек.....	11
3.2	Избор на език за програмиране: Python	12
3.3	Сървърна архитектура: Django и Django REST Framework (DRF).....	12
3.4	Библиотека за хардуерна емуляция: PyAutoGUI	15
3.5	Клиентска част (Frontend): Vue.js, Axios и HTML/CSS.....	16
4.	АРХИТЕКТУРА И РЕАЛИЗАЦИЯ НА СОФТУЕРНОТО РЕШЕНИЕ.....	18
4.1	Структура на проекта и базови конфигурации	18
4.2	Разработка на сървърната логика (Backend)	18
4.2.1	Изграждане на API крайни точки (Endpoints)	18
4.2.2	Маршрутизиране на заявките (Routing)	20
4.3	Изграждане на потребителския интерфейс (Frontend)	20
4.3.1	Визуален дизайн и потребителско изживяване (UX).....	20

4.3.2	Асинхронна логика с Vue.js и Axios.....	21
4.4	Мрежова конфигурация и интеграция.....	22
5.	ТЕСТВАНЕ, РЕЗУЛТАТИ И ПРИНОСИ.....	23
5.1	Методология на тестването.....	23
5.1.1	Фаза 1: Локално тестване (Localhost).....	23
5.1.2	Фаза 2: Мрежово тестване с мобилно устройство	24
5.2	Отстраняване на мрежови проблеми (Troubleshooting)	25
5.3	Резултати от практическото приложение.....	26
5.4	Анализ на изпълнението на задачите.....	27
5.5	Приноси на дипломния проект.....	27
6.	ЗАКЛЮЧЕНИЕ	28
7.	Литература:	30

1. Увод

Бурното развитие на информационните и комуникационните технологии през последното десетилетие доведе до повсеместното навлизане на мобилните устройства във всяка сфера на съвременния живот. Смартфоните отдавна престанаха да бъдат просто средство за гласова комуникация – те се превърнаха в мощни преносими компютри, оборудвани с високоскоростна интернет свързаност, интуитивни сензорни екрани и възможност за изпълнение на сложни уеб базирани приложения. Успоредно с това, концепцията за "интернет на нещата" (Internet of Things - IoT) и свързаността между различни устройства в локални мрежи открива нови хоризонти за автоматизация и дистанционно управление.

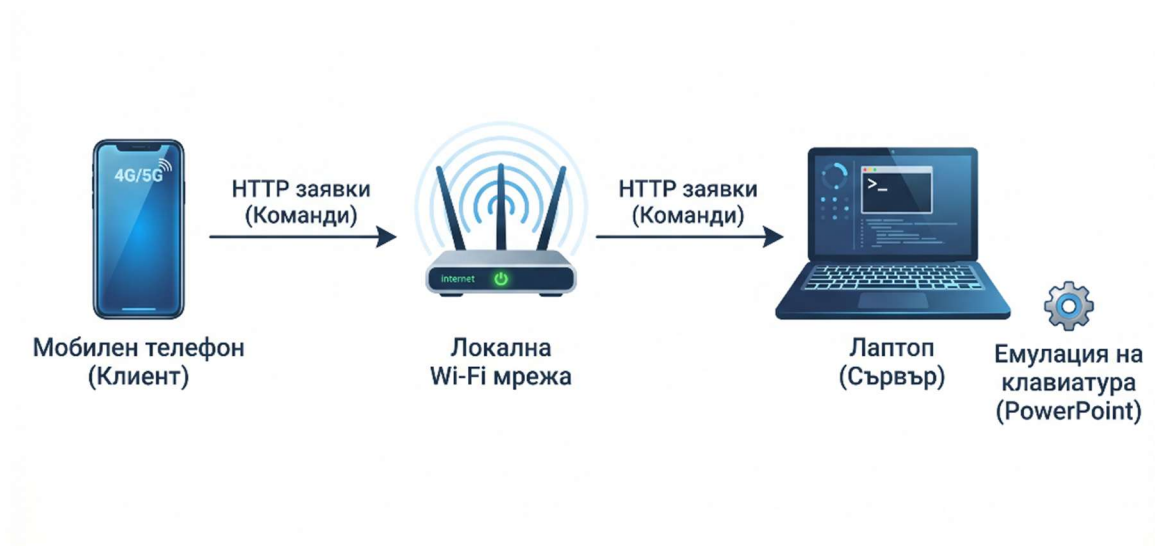
Въпреки тази технологична еволюция, в редица ежедневни ситуации потребителите продължават да бъдат физически обвързани със своите персонални компютри или лаптопи. Типичен пример за това е изнасянето на презентации или управлението на мултимедийни десктоп приложения пред публика. Често презентаторът е принуден да стои плътно до клавиатурата на лаптопа, за да сменя слайдовете, или да разчита на допълнителен, специализиран хардуер – т.нар. "презентъри" или "кликъри", които изискват допълнителна инвестиция и носят риск от техническа неизправност (изтощени батерии, загуба на USB приемник и др.).

Именно тук се поражда необходимостта от създаване на гъвкаво, софтуерно базирано решение, което да използва наличния в джоба на всеки потребител хардуер – неговия личен смартфон.

Целта на настоящия дипломен проект е да се проектира, разработи и внедри система от тип "клиент-сървър", която да позволява дистанционно управление на десктоп приложение (с фокус върху Microsoft PowerPoint) чрез мобилен телефон. Системата трябва да осигури надеждна комуникация в рамките на локална безжична мрежа (Wi-Fi), като превърне сензорния екран на телефона в удобно дистанционно управление, без необходимост от инсталиране на допълнителни мобилни приложения (Native Apps) от магазини като Google Play или App Store.

За постигането на тази основна цел, проектът си поставя следните конкретни задачи и очаквани резултати:

- Проектиране на сървърна архитектура: Изграждане на надежден локален уеб сървър, базиран на езика Python и рамката Django, който да приема асинхронни HTTP заявки.
- Разработка на приложно-програмен интерфейс (API): Интегриране на Django REST Framework (DRF) за създаване на API крайни точки (endpoints), които да обработват команди в реално време.
- Емуляция на хардуерни събития: Внедряване на логика, чрез която сървърът да "превежда" получените мрежови заявки в реални натискания на клавиши от клавиатурата на лаптопа (емулация на стрелки за навигация, функционални клавиши като F5 и Esc) чрез специализирани библиотеки.
- Изграждане на потребителски интерфейс (Frontend): Създаване на адаптивно (responsive) уеб приложение, достъпно през брауъра на телефона. Интерфейсът трябва да е изграден с помощта на съвременни технологии като Vue.js и Bootstrap, за да осигури големи, удобни за натискане бутони ("Напред", "Назад", "Старт" и др.) и визуална обратна връзка (статус) за успеваемостта на всяка команда.
- Осигуряване на сигурност и обработка на грешки: Имплементиране на базова защита при комуникацията (работа с CSRF токени) и адекватно управление на изключения при липса на връзка със сървъра или неразпознати команди.



Фигура 1. Блок-схема на комуникацията между мобилния телефон (клиент) и лаптопа (сървър) през локалната Wi-Fi мрежа

Реализацията на този проект не само решава конкретен практически проблем, но и служи като отлична платформа за демонстрация на съвременни концепции в уеб разработката – обединяване на Backend и Frontend технологии, работа с мрежови протоколи и системна автоматизация на ниво операционна система. В следващите глави на дипломната работа ще бъде направен детайлен преглед на използваните технологии, архитектурата на софтуерното решение и практическите стъпки по неговото внедряване и тестване.

2. АНАЛИЗ НА ПРОБЛЕМА И СЪЩЕСТВУВАЩИТЕ РЕШЕНИЯ

2.1 ДЕФИНИРАНЕ И СЪЩНОСТ НА ПРОБЛЕМА

В съвременната динамична работна и образователна среда, представянето на информация пред публика (презентирането) е ключов елемент от комуникацията. Традиционният модел на презентиране изисква лекторът да управлява визуалния материал (най-често слайдове в Microsoft PowerPoint, Google Slides или друг софтуер) чрез директно взаимодействие с компютърната периферия – клавиатура или мишка. Това създава физическа бариера и ограничава мобилността на презентатора, като го принуждава да остане статичен зад банка или бюро. Статичното позициониране от своя страна намалява зрителния контакт с аудиторията, ограничава езика на тялото и като цяло понижава ангажираността на слушателите.

За преодоляване на този проблем, технологичната индустрия предлага различни решения, които обаче често носят своите собствени недостатъци. Основното предизвикателство, което настоящият дипломен проект адресира, е липсата на универсален, безплатен и независим от платформата метод за дистанционно управление на десктоп приложения, който да не изисква инсталация на допълнителен софтуер от страна на клиента (смартфона).

Проблемът може да бъде разгледан в три основни аспекта:

- **Хардуерен аспект:** Необходимост от допълнителни устройства, които изискват поддръжка (смяна на батерии) и носят риск от хардуерна несъвместимост или загуба на свързващи компоненти (USB приемници).
- **Софтуерен аспект:** Зависимост от конкретни операционни системи (Android, iOS, Windows, macOS). Повечето съществуващи софтуерни решения изискват инсталиране на "Native" (нативни) приложения и на двете комуникиращи устройства.
- **Мрежов аспект:** Нужда от надеждна и бърза комуникация (с минимално закъснение – latency) между управляващото и управляваното устройство в рамките на локална мрежа.

2.2 АНАЛИЗ НА АРХИТЕКТУРАТА „КЛИЕНТ-СЪРВЪР“ В КОНТЕКСТА НА ПРОЕКТА

За да се реализира ефективно дистанционно управление, системата трябва да бъде изградена върху класическата мрежова архитектура "Клиент-Сървър" (Client-Server model).



Фигура 2. Архитектурен модел Клиент-Сървър през локална Wi-Fi мрежа, показващ посоката на HTTP заявките и отговорите.

В рамките на тази разработка, ролите са разпределени по следния начин:

- **Сървър (Server):** Това е преносимият или настолният компютър (лаптоп), на който се изпълнява десктоп приложението (напр. PowerPoint). На този компютър се стартира локален уеб сървър, разработен на Python/Django. Сървърът е в

състояние на постоянно "слушане" (listening) за входящи мрежови заявки. Неговата основна задача е да приема командите, да ги валидира и да ги транслира в системни събития (емулация на натискане на клавиш).

- **Клиент (Client):** Това е мобилното устройство (смартфон), което инициира комуникацията. Клиентът осъществява достъп до сървъра чрез стандартен уеб браузър (Chrome, Safari и др.), зареждайки графичния потребителски интерфейс (Frontend). При взаимодействие на потребителя с интерфейса (натискане на бутон), клиентът генерира и изпраща HTTP заявка към сървъра.

Комуникацията се осъществява чрез обмен на структурирани данни, най-често във формат JSON (JavaScript Object Notation).

Например, когато потребителят натисне бутона за следващ слайд, клиентът изпраща към сървъра следния примерен JSON полезен товар (payload):

```
json
{
  "action": "next_slide",
  "timestamp": "2026-04-18T10:30:00Z"
}
```

Сървърът анализира полето "action", разпознава командата "next_slide" и чрез подходяща библиотека подава сигнал към операционната система да симулира натискане на клавиша Right Arrow (Стрелка надясно).

2.3 ПРЕГЛЕД НА СЪЩЕСТВУВАЩИТЕ РЕШЕНИЯ И ТЕХНИТЕ ОГРАНИЧЕНИЯ

Преди пристъпване към реализацията на собствено решение, е направен обстоен анализ на съществуващите алтернативи на пазара. Те могат да бъдат класифицирани в две основни категории:

A. Хардуерни презентатори (Clickers/Presenters)

Това са специализирани периферни устройства, които комуникират с компютъра посредством радиочестотен сигнал (обикновено 2.4 GHz чрез USB донгъл) или Bluetooth.

- **Предимства:** Изключително надеждни, с ниско времезакъснение. Често разполагат с физически бутони, които лекторът може да напипва без да гледа, както и с вградена лазерна показалка.
- **Недостатъци:** Изискват финансова инвестиция. Зависят от батерии, които могат да се изтопят по време на презентация. USB приемниците са малки и лесно се губят. Не предлагат гъвкавост за добавяне на нови функции, тъй като хардуерът е предварително програмиран за конкретни клавишни комбинации (обикновено Page Up / Page Down).

Б. Специализирани мобилни приложения (Native Apps)

Примери за такива решения са приложения като Unified Remote, PowerPoint Remote или PC Remote, които се инсталират от магазините за мобилни приложения.

- **Предимства:** Предлагат богата функционалност (управление на мишка, клавиатура, файлов мениджър).
- **Недостатъци:** Изискват инсталационен процес както на компютъра (сървърна програма), така и на телефона (клиентско приложение). Това създава значително неудобство (т.нар. friction), особено ако лекторът трябва да презентира от чужд компютър, на който няма администраторски права за инсталиране на софтуер. Освен това, разработчиците на тези приложения трябва да поддържат различни версии за Android и iOS, което оскъпява и усложнява поддръжката.

2.4 ФОРМУЛИРАНЕ НА ИЗИСКВАНИЯТА КЪМ СОФТУЕРНАТА СИСТЕМА

Въз основа на направения анализ, бе взето решение да се разработи Уеб-базирано решение за дистанционно управление. Този подход комбинира предимствата на мобилните технологии с универсалността на уеб браузърите.

За да бъде проектът успешен и да преодолее недостатъците на съществуващите системи, той трябва да отговаря на следните системни изисквания:

- Нулево инсталиране на клиента: Системата трябва да е достъпна единствено чрез въвеждане на IP адрес в уеб браузъра на телефона (в концепцията на Bring Your Own Device - BYOD).
- Кросплатформеност: Интерфейсът трябва да работи безупречно независимо от операционната система на телефона (Android, iOS) или размера на екрана.

- Асинхронност: Комуникацията трябва да се случва във фонов режим (чрез AJAX/Axios заявки), без да се презарежда страницата на телефона при всяко натискане на бутон.
- Модулност на сървъра: Логиката за приемане на мрежови заявки трябва да е отделена от логиката за хардуерна емуляция, за да може в бъдеще системата да се разшири за управление на други програми (напр. медийни плейъри).

Тези изисквания обосновават избора на конкретния технологичен стек (Python, Django, DRF, Vue.js), който ще бъде детайлно разгледан в следващата глава.

2.5 ТЕОРЕТИЧНИ ОСНОВИ НА МРЕЖОВАТА КОМУНИКАЦИЯ В ЛОКАЛНА СРЕДА (LAN)

За да се осъществи дистанционното управление, предложената система разчита на локална компютърна мрежа (Local Area Network - LAN), базирана на стандарта IEEE



802.11 (Wi-Fi). За разлика от облачните (Cloud) решения, при които заявките преминават през външни сървъри и интернет доставчици (ISP), комуникацията в LAN среда се осъществява на ниво интранет.

Този подход има сериозна теоретична обосновка от гледна точка на мрежовия модел OSI (Open Systems Interconnection):

Мрежов слой (Слой 3):

Използва се протоколът IPv4. Устройствата (телефон и лаптоп) получават локални IP адреси (най-често в диапазона 192.168.x.x или 10.x.x.x), назначени динамично чрез DHCP протокол от безжичния рутер. Тъй като пакетите с данни не напускат локалния мрежов сегмент (Subnet), не се извършва мрежова трансляция на адреси (NAT), което драстично намалява времето за маршрутизиране.

Транспортен слой (Слой 4): Комуникацията се базира на Transmission Control Protocol (TCP). За разлика от UDP, TCP е протокол, ориентиран към свързване, който гарантира доставката на пакетите в правилния ред. За система за управление това е критично – натискането на бутон за следващ слайд не бива да се "губи" по мрежата.

Приложен слой (Слой 7): Данните се пренасят чрез HTTP (Hypertext Transfer Protocol). В контекста на проекта се използват HTTP POST заявки, които носят полезен товар (payload) с инструкции.

Следователно, теоретичното предимство на LAN-базираната комуникация е свеждането на мрежовата латентност (latency) до теоретичния минимум (често под 5-10 ms), елиминирането на зависимостта от външна интернет свързаност и повишаването на поверителността на данните.

2.6 ПРИНЦИПИ НА ВЗАИМОДЕЙСТВИЕТО ЧОВЕК-КОМПЮТЪР (HUMAN-COMPUTER INTERACTION - HCI)

Един от основните теоретични проблеми при изграждането на дистанционни управления е когнитивното натоварване (Cognitive Load) на презентатора. По време на презентация вниманието на лектора е насочено към публиката и изложената материя. Ако интерфейсът изисква лекторът да поглежда екрана на телефона си, за да "нацели" малък бутон, това прекъсва зрителния контакт с аудиторията.

В тази връзка, дизайнът на предложеното мобилно приложение стъпва на Закона на Фитс (Fitts's Law). Това е фундаментален модел от ергономията и HCI, който гласи, че времето, необходимо за бързо преместване до дадена цел (бутон), е функция от съотношението между разстоянието до целта и нейната ширина.

Математическият модел на Закона на Фитс се изразява като:

$$T = a + b \cdot \log_2 \left(\frac{D}{W} + 1 \right)$$

Където:

- T е времето за извършване на движението.
- b са емпирично определени константи.
- D е разстоянието от началната точка до центъра на целта.
- W е ширината на целта (бутона).

Според тази формула, за да се минимизира времето T (и съответно усилията на потребителя), бутоните трябва да бъдат възможно най-големи (W да клони към максимума на екрана). Именно тази теоретична постановка обуславя дизайна на потребителския интерфейс (UI) в проекта – създаването на масивни бутони на цял екран, които заемат огромна активна площ (hitbox), позволявайки на презентатора да оперира със системата чрез периферно зрение или тактилна памет, без да отделя поглед от публиката.

3. ОПИСАНИЕ И АРГУМЕНТАЦИЯ НА ИЗБОРА НА ТЕХНОЛОГИИ

3.1 ВЪВЕДЕНИЕ В ТЕХНОЛОГИЧНИЯ СТЕК

Изграждането на надеждна система за дистанционно управление изисква внимателен подбор на софтуерни инструменти, които да осигурят безпроблемна комуникация между разнородни устройства (смартфон и персонален компютър). За реализацията на настоящия дипломен проект е избран модерен технологичен стек, базиран на уеб технологии, който разделя системата на две основни логически части: сървърна (Backend) и клиентска (Frontend).

Този подход, известен като отделяне на отговорностите (Separation of Concerns), позволява независима разработка и тестване на компонентите. Избраните технологии са с отворен код (Open Source), притежават богата документация и се поддържат от големи общности от разработчици, което гарантира устойчивостта на проекта.

3.2 ИЗБОР НА ЕЗИК ЗА ПРОГРАМИРАНЕ: PYTHON

В основата на сървърната логика и хардуерната емуляция стои езикът за програмиране Python. Изборът му не е случаен и се базира на няколко ключови предимства в контекста на системната интеграция:

- **Високо ниво на абстракция и четимост:** Синтаксисът на Python е изключително изчистен, което улеснява бързото прототипиране и намалява времето за разработка (Rapid Application Development).
- **Богата стандартна библиотека и екосистема:** Python разполага с огромен брой външни пакети (управлявани чрез мениджъра pip), които покриват широк спектър от задачи – от изграждане на уеб сървъри до директно взаимодействие с операционната система (ОС).
- **Кросплатформеност:** Скриптовете, написани на Python, могат да се изпълняват без модификация върху различни операционни системи (Windows, macOS, Linux), стига да е наличен съответният интерпретатор. Това прави сървърната част на проекта преносима.

3.3 СЪРВЪРНА АРХИТЕКТУРА: DJANGO И DJANGO REST FRAMEWORK (DRF)

За изграждането на сървърната част, която да обработва мрежовите заявки, е избрана уеб рамката (framework) Django. Тя следва архитектурния модел MVT (Model-View-Template) и предоставя готови решения за маршрутизиране (routing), сигурност и обработка на HTTP заявки.

Въпреки че Django предлага собствена система за визуализиране на HTML страници, спецификата на проекта изисква създаването на приложно-програмен интерфейс (API), чрез който мобилното устройство да изпраща команди асинхронно, без да презарежда страницата. За тази цел към основния проект е интегрирана библиотеката Django REST Framework (DRF).

Предимства на DRF за проекта:

- Осигурява лесно създаване на API крайни точки (endpoints).
- Автоматично обработва сериализацията и десериализацията на данни (преобразуване на Python обекти в JSON формат и обратно).

- Предоставя вградени механизми за валидация на входните данни, което предпазва сървъра от некоректни или злонамерени заявки.

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework', # Интеграция на DRF
    'main', # Основното приложение на проекта
]
```

3.3.1 Теоретичен анализ на архитектурния стил REST (Representational State Transfer)

Внедряването на Django REST Framework (DRF) в проекта не е просто практическо удобство, а следване на строг софтуерно-архитектурен стил. REST архитектурата, дефинирана от Рой Филдинг, налага серия от ограничения (constraints), които правят системата мащабируема и независима:

- **Клиент-сървър (Client-Server):** Пълно отделяне на потребителския интерфейс (Vue.js на телефона) от съхранението на данни и логиката (Django на лаптопа). Това позволява двата компонента да се развиват независимо.
- **Безсъстоятелност (Statelessness):** Всяка HTTP заявка от телефона към лаптопа трябва да съдържа цялата необходима информация за нейното разбиране (напр. точната команда `next_slide`). Сървърът не пази "състояние" (session state) за това кой слайд е текущ. Той е просто изпълнител. Операционната система (PowerPoint) е тази, която пази състоянието на презентацията.
- **Единен интерфейс (Uniform Interface):** Комуникацията става чрез стандартизиран формат – в нашия случай JSON (JavaScript Object Notation). JSON е лек текстово-базиран формат, който е езиково независим, което означава, че JavaScript (клиентът) и Python (сървърът) го парсират с минимален разход на изчислителен ресурс.

Изборът на REST пред алтернативни технологии като WebSockets (които поддържат постоянна двупосочна връзка) е теоретично обоснован от факта, че дистанционното управление генерира дискретни, асинхронни събития. Поддържането на отворен WebSocket би консумирало излишна памет на лаптопа, докато REST

моделът "заявка-отговор" е оптимален за спорадично изпращане на команди (натискане на бутон през няколко минути).

3.4 ТЕОРЕТИЧНИ АСПЕКТИ НА УЕБ СИГУРНОСТТА (CSRF)

Всяка уеб-базирана комуникация, дори и в локална мрежа, е уязвима на специфични вектори на атака. Разработената система интегрира фундаментални защитни механизми, като основният от тях е защитата срещу CSRF (Cross-Site Request Forgery).

CSRF е тип атака, при която злонамерен софтуер или сайт принуждава браузъра на потребителя да изпълни нежелано действие в уеб приложение, в което потребителят вече е автентикиран или свързан. В контекста на проекта, ако защитата липсваше, злонамерен скрипт от друга отворена страница на телефона би могъл да изпрати POST заявка към лаптопа и неконтролируемо да сменя слайдовете.

За да се предотврати това, Django налага стриктна валидация чрез CSRF токени. Теорията на този механизъм се базира на криптографски синхронизатор (Synchronizer Token Pattern):

Когато телефонът зареди началната страница, сървърът генерира уникален, псевдослучаен низ (токен) и го изпраща като бисквитка (cookie).

При всеки опит за POST заявка (напр. натискане на бутон "Напред"), JavaScript логиката (Axios) трябва да прочете тази бисквитка и да я постави в специален HTTP Header (X-CSRFToken).

Сървърът сравнява токена от хедъра с този, който очаква. Ако съвпадат, заявката се изпълнява. В противен случай сървърът връща грешка 403 Forbidden.

Този процес демонстрира спазването на принципа на "Защита в дълбочина" (Defense in Depth) при проектирането на съвременни софтуерни системи.

3.5 ТЕОРИЯ НА ХАРДУЕРНАТА ЕМУЛАЦИЯ (OS LEVEL INTERRUPTS)

Използването на библиотеката PyAutoGUI повдига интересни теоретични въпроси относно това как софтуер може да "симулира" хардуер.

В класическата компютърна архитектура, когато потребител натисне физически клавиш, клавиатурата изпраща хардуерно прекъсване (Hardware Interrupt) към процесора чрез контролера на прекъсванията (IRQ). Операционната система прихваща това прекъсване и го преобразува в събитие (Event), което се изпраща към активното на преден план (focused) приложение.

PyAutoGUI заобикаля хардуерния слой и инжектира събития директно в API-то на операционната система (например SendInput API при Windows или XTest при Linux). Теоретично, операционната система (и съответно PowerPoint) не може да направи разлика между събитие, генерирано от реална физическа клавиатура, и такова, инжектирано софтуерно. Това прави решението универсално и гарантира 100% съвместимост с всяко приложение, което приема вход от клавиатурата.

3.6 БИБЛИОТЕКА ЗА ХАРДУЕРНА ЕМУЛАЦИЯ: PYAUTOGUI

Един от най-критичните модули в проекта е "преводачът" между уеб заявките и реалните действия на компютъра. Тъй като уеб сървърът (Django) работи в изолирана среда, той се нуждае от специализиран инструмент, за да симулира физическо натискане на клавиши (например стрелките за смяна на слайдове в PowerPoint).

За тази цел е използвана библиотеката PyAutoGUI. Тя е кросплатформен модул за автоматизация на графичния потребителски интерфейс (GUI automation).

- **Функционалност:**

PyAutoGUI може програмно да контролира мишката и клавиатурата. В контекста на заданието, функциите `pyautogui.press('right')` и `pyautogui.press('left')` се използват за симулиране на натискане на дясна и лява стрелка.

- **Безопасност:**

Библиотеката разполага с вграден механизъм за защита (Fail-Safe), който прекратява изпълнението на скрипта, ако мишката бъде преместена в някой от ъглите на екрана, предотвратявайки загуба на контрол над системата.

3.7 КЛИЕНТСКА ЧАСТ (FRONTEND): VUE.JS, AXIOS И HTML/CSS

За да бъде системата удобна за използване, мобилният интерфейс трябва да бъде интуитивен, бърз и отзивчив. Според изискванията на проекта, клиентската част е реализирана в единствен HTML темплейт (Single Page Application подход), което елиминира нуждата от сложни инсталации.

За реализиране на динамичното поведение е избран JavaScript фреймворк Vue.js.

Защо Vue.js? Vue.js е прогресивен фреймвърк, който позволява лесна интеграция директно чрез CDN (Content Delivery Network), без необходимост от сложна конфигурация на системи за сглобяване (като Webpack или Vite). Тя предлага реактивност – автоматично обновяване на потребителския интерфейс при промяна на състоянието на приложението (например, показване на съобщение "Слайдът е сменен" веднага след успешно изпълнена команда).

За мрежовата комуникация между Vue.js и Django сървър се използва библиотеката Axios.

Ролята на Axios: Axios е базиран на обещания (Promise-based) HTTP клиент за браузър. Той се използва за изпращане на асинхронни POST заявки към DRF крайните точки. Когато потребителят натисне бутон на телефона си, Axios "опакова" командата в JSON формат и я изпраща към сървър във фонов режим. Това елиминира "премигването" на екрана и осигурява плавно потребителско изживяване, сравнимо с това на обикновено мобилно приложение.

```
<!-- Тук започва Vue.js приложението -->
<div id="app" class="container mt-4 text-center">
  <h2 class="app-title">📄 РРТ Контролер</h2>

  <!-- Статус съобщение -->
  <div style="height: 30px; margin-bottom: 20px;">
    <span :class="statusClass">{{ statusMessage }}</span>
  </div>
```



```

<div class="row g-3">
  <!-- Бутон за старт на презентацията -->
  <div class="col-12">
    <button @click="sendCommand('start')" class="btn btn-
success w-100 btn-huge">
      ▶ Старт (F5)
    </button>
  </div>

  <!-- Бутони за навигация -->
  <div class="col-6">
    <button @click="sendCommand('prev')" class="btn btn-
primary w-100 btn-huge">
      ◀ Назад
    </button>
  </div>
  <div class="col-6">
    <button @click="sendCommand('next')" class="btn btn-
primary w-100 btn-huge">
      Напред ▶
    </button>
  </div>

  <!-- Допълнителни контроли -->
  <div class="col-6 mt-4">
    <button @click="sendCommand('black')" class="btn btn-dark
btn-circle border border-secondary">
      Черен<br>Екран
    </button>
  </div>
  <div class="col-6 mt-4">
    <button @click="sendCommand('white')" class="btn btn-
light btn-circle border border-secondary text-dark">
      Бял<br>Екран
    </button>
  </div>

  <!-- Бутон за изход -->
  <div class="col-12 mt-4">
    <button @click="sendCommand('esc')" class="btn btn-danger
w-100 btn-huge" style="height: 60px; font-size: 1.2rem;">
      ◻ Край (Esc)
    </button>
  </div>
</div>
</div>

<!-- Vue 3 CDN -->
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
<!-- Axios CDN -->
<script
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>

```

Комбинацията от тези технологии създава изключително гъвкава, лека и ефективна система. Django и DRF осигуряват стабилен гръбнак и сигурност, PyAutoGUI осъществява директния контрол над десктоп средата, а Vue.js и Axios предоставят модерен, асинхронен интерфейс за крайния потребител. В следващата

глава ще бъде разгледана конкретната практическа реализация и интеграцията на тези компоненти в работещ софтуерен продукт.

4. АРХИТЕКТУРА И РЕАЛИЗАЦИЯ НА СОФТУЕРНОТО РЕШЕНИЕ

4.1 СТРУКТУРА НА ПРОЕКТА И БАЗОВИ КОНФИГУРАЦИИ

Практическата реализация на дипломния проект започва със създаването на базов проект в средата на езика Python чрез рамката Django. Проектът е наименуван DP (съкращение от Дипломен Проект), а в него е създадено основно приложение (app), носещо името main.

За да бъде изградена необходимата среда, предварително са инсталирани нужните библиотеки чрез пакетния мениджър pip. Основните зависимости включват:

- `django`restframework – за изграждане на приложно-програмния интерфейс (API).
- `pyautogui` – за осъществяване на хардуерната емуляция на клавиши.

След успешната инсталация, библиотеката `rest_framework`, както и самото приложение main, са регистрирани в основния конфигурационен файл на проекта (`DP/settings.py`) в списъка `INSTALLED_APPS`, което гарантира правилното им функциониране в екосистемата на Django.

4.2 РАЗРАБОТКА НА СЪРВЪРНАТА ЛОГИКА (BACKEND)

Ядрото на системата за управление е сървърната логика, която приема мрежовите заявки и ги преобразува във физически действия.

4.2.1 Изграждане на API крайни точки (Endpoints)

Във файла `main/views.py` са реализирани два основни изгледа. Първият е стандартен изглед `index`, който отговаря за сервирането на графичния интерфейс към клиента. Вторият, и по-съществен, е API endpoint-ът `send_command`, който е декориран

с `@api_view(['POST'])` от DRF. Това гарантира, че функцията ще приема единствено POST заявки, съдържащи команди за изпълнение.

```
# main/views.py
from django.shortcuts import render
from rest_framework.decorators import api_view
from rest_framework.response import Response
import pyautogui

# Това е изгледът за твоя бъдещ HTML/Vue.js frontend
def index(request):
    return render(request, 'main/index.html')

# Това е API endpoint-ът, който ще приема командите от телефона
@api_view(['POST'])
def send_command(request):
    # Вземаме командата от тялото на POST заявката
    command = request.data.get('action')

    # Речник, който свързва нашите команди (от телефона) с реални
    # клавиши за PowerPoint
    commands_map = {
        'next': 'space',
        'prev': 'left',
        'start': 'f5',
        'esc': 'escape',
        'black': 'b',
        'white': 'w'
    }

    if command in commands_map:
        key_to_press = commands_map[command]
        try:
            # Симулираме натискане на клавиш
            pyautogui.press(key_to_press)
            return Response({"status": "success", "message":
f"Изпълнена команда: {command}"}, status=200)
        except Exception as e:
            # В случай на грешка с операционната система
            return Response({"status": "error", "message": str(e)},
status=500)
    else:
        return Response({"status": "error", "message": "Непозната
команда"}, status=400)
```

Както се вижда от кода, системата използва речник (`commands_map`), който служи като преводач. Например, ако телефонът изпрати команда `start`, сървърът намира съответствието `f5` и подава сигнал към операционната система да симулира натискане на клавиша за стартиране на презентация в PowerPoint. Блокът `try-except` гарантира, че дори при възникване на системна грешка, сървърът няма да прекъсне работата си, а ще върне адекватен статус код 500 на клиента.

4.2.2 Маршрутизиране на заявките (Routing)

За да могат заявките от телефона да достигнат до правилната функция, е изградена система за маршрутизиране. Във файла `main/urls.py` са дефинирани точните URL адреси:

```
# main/urls.py
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static
from . import views

urlpatterns = [
    # Път към основната страница (Frontend-a)
    path('', views.index, name='index'),

    # Път към нашето API за приемане на команди
    path('api/command/', views.send_command, name='send_command'),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
                           document_root=settings.MEDIA_ROOT)
```

По този начин, когато клиентът изпрати POST заявка към `/api/command/`, Django автоматично я насочва към функцията `send_command`.

4.3 ИЗГРАЖДАНЕ НА ПОТРЕБИТЕЛСКИЯ ИНТЕРФЕЙС (FRONTEND)

Клиентската част е реализирана като Single Page Application (Едностранично приложение) в рамките на единствен HTML темплейт (`index.html`), разположен в папката `templates` на приложението `main`.

4.3.1 Визуален дизайн и потребителско изживяване (UX)

За бързото и ефективно стилизиране е използвана библиотеката Bootstrap 5, заредена чрез CDN (Content Delivery Network). Дизайнът следва принципите на "Mobile-First" подхода. Предвид факта, че презентаторът често гледа към публиката, а не към екрана на телефона си, интерфейсът е изграден с тъмна тема (`background-color: #2c3e50`), която пести батерия, и използва персонализирани CSS класове като `.btn-huge`. Бутоните заемат цялата ширина на екрана, имат голям шрифт (1.5rem) и височина от 80 пиксела, което минимизира шанса за грешно натискане.



Фигура 3. Изглед на контролния панел на мобилното устройство, показващ масивните бутони за навигация (Старт, Напред, Назад, Край) и статус индикатора.

4.3.2 Асинхронна логика с Vue.js и Axios

Динамичното поведение на страницата се управлява от Vue 3. Всяко натискане на бутон извиква метода `sendcommand(action)`. Системата интегрира механизъм за сигурност, като първо извлича CSRF (Cross-Site Request Forgery) токена от бисквитките на браузъра чрез специална функция `getCookie`.

След извличане на токена, библиотеката `Axios` изпраща асинхронна заявка към сървъра.

```
// Основната функция, която праща заявката към сървъра
sendCommand(action) {
  const csrftoken = this.getCookie('csrftoken');

  // Показваме, че зареждаме
  this.statusMessage = `Изпращане на '${action}'...`;
  this.statusClass = 'text-warning';

  // Axios POST заявка към нашия API endpoint
  axios.post('/api/command/', { action: action }, {
    headers: {
      'X-CSRFToken': csrftoken, // Изпращаме токена за
      'Content-Type': 'application/json'
    }
  })
  .then(response => {
    // При успех
    this.statusMessage = `✅ Успешно: ${action}`;
    this.statusClass = 'text-success';
  })
}
```

```

        // Изчистваме съобщението след 2 секунди
        setTimeout(() => {
            this.statusMessage = 'Готов за команди';
            this.statusClass = 'text-info';
        }, 2000);
    })
    .catch(error => {
        // При грешка
        this.statusMessage = '✗ Грешка при връзката!';
        this.statusClass = 'text-danger';
        console.error(error);
    });
}

```

Този фрагмент от кода е ключов за потребителското изживяване. Той осигурява визуална обратна връзка (feedback) в реално време – променя текста и цвета на статус индикатора (от жълто при изпращане, към зелено при успех или червено при грешка), без да презарежда страницата.

4.4 МРЕЖОВА КОНФИГУРАЦИЯ И ИНТЕГРАЦИЯ

За да се осъществи реалната комуникация между телефона и лаптопа, и двете устройства трябва да бъдат свързани към една и съща локална Wi-Fi мрежа.

Системата е конфигурирана за мрежови достъп чрез две основни стъпки:

Разрешаване на външни хостове: Във файла `settings.py` променливата `ALLOWED_HOSTS` е настроена да приема заявки от локалния IP адрес на компютъра (напр. `['192.168.1.5', '127.0.0.1']`) или глобално за локалната мрежа (`['*']`).

Стартиране на сървъра в мрежов режим: За разлика от стандартното стартиране на Django сървъра (който слуша само на `localhost`), проектът се стартира чрез командата `python manage.py runserver 0.0.0.0:8000`. Адресът `0.0.0.0` инструктира сървъра да приема входящи мрежови връзки от всички мрежови интерфейси на машината.

```
(.venv) PS D:\DZI_2026\SP\Theme_12_Martin\DP> python manage.py runserver 0.0.0.0:8800
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 19, 2026 - 00:13:38
Django version 4.2.18, using settings 'DP.settings'
Starting development server at http://0.0.0.0:8800/
Quit the server with CTRL-BREAK.

[19/Apr/2026 00:13:57] "GET / HTTP/1.1" 200 6311
```

Фигура 4. Стартиране на Django сървър и успешно приемане на мрежови заявки (HTTP 200 OK) от мобилното устройство.

След изпълнението на тази конфигурация, потребителят трябва единствено да въведе локалния IP адрес на лаптопа и порта (напр. <http://192.168.1.5:8000>) в брауъра на своя телефон, за да зареди контролния интерфейс и да започне управлението на презентацията.

5. ТЕСТВАНЕ, РЕЗУЛТАТИ И ПРИНОСИ

5.1 МЕТОДОЛОГИЯ НА ТЕСТВАНЕТО

За да се гарантира надеждността и стабилността на разработената система за дистанционно управление, беше приложен итеративен подход на тестване в реална среда. Тестването премина през две основни фази: локално тестване на сървъра (Localhost) и мрежово тестване в реална безжична мрежа (WLAN).

5.1.1 Фаза 1: Локално тестване (Localhost)

В началния етап системата беше стартирана и тествана изцяло на машината, изпълняваща ролята на сървър (лаптопа). Целта на тази фаза беше да се верифицира, че маршрутизирането в Django работи коректно и че библиотеката PyAutoGUI успешно симулира хардуерни събития. За целта, уеб интерфейсът беше зареден в брауъра на самия лаптоп (на адрес <http://127.0.0.1:8000>).

Установено бе, че при натискане на виртуалните бутони в брауъра, операционната система регистрира съответните клавишни комбинации (напр.

натискането на бутона "Старт" успешно стартира отворен в момента файл на PowerPoint чрез симулиране на клавиша F5).

5.1.2 Фаза 2: Мрежово тестване с мобилно устройство

Втората и най-критична фаза обхвана реалната комуникация между смартфона и лаптопа. И двете устройства бяха свързани към обща локална Wi-Fi мрежа. Сървърът беше стартиран да слуша на всички интерфейси чрез командата `python manage.py runserver 0.0.0.0:8000`.

За да се осъществи връзката, беше необходимо да се установи локалният IPv4 адрес на сървъра. Това се постигна чрез изпълнение на мрежовата команда в терминала на Windows:

```
Ethernet adapter Ethernet 3:
```

```
Connection-specific DNS Suffix . :  
Link-local IPv6 Address . . . . . : fe80::fe03:4412:34c5:f6d9%13  
IPv4 Address. . . . . : 192.168.1.200  
Subnet Mask . . . . . : 255.255.255.0  
IPv4 Address. . . . . : 192.168.2.200  
Subnet Mask . . . . . : 255.255.255.0  
IPv4 Address. . . . . : 192.168.100.200  
Subnet Mask . . . . . : 255.255.255.0  
Default Gateway . . . . . : 192.168.100.1
```

```
Ethernet adapter Bluetooth мрежова връзка 2:
```

```
Media State . . . . . : Media disconnected  
Connection-specific DNS Suffix . :
```

```
Ethernet adapter vEthernet (Default Switch):
```

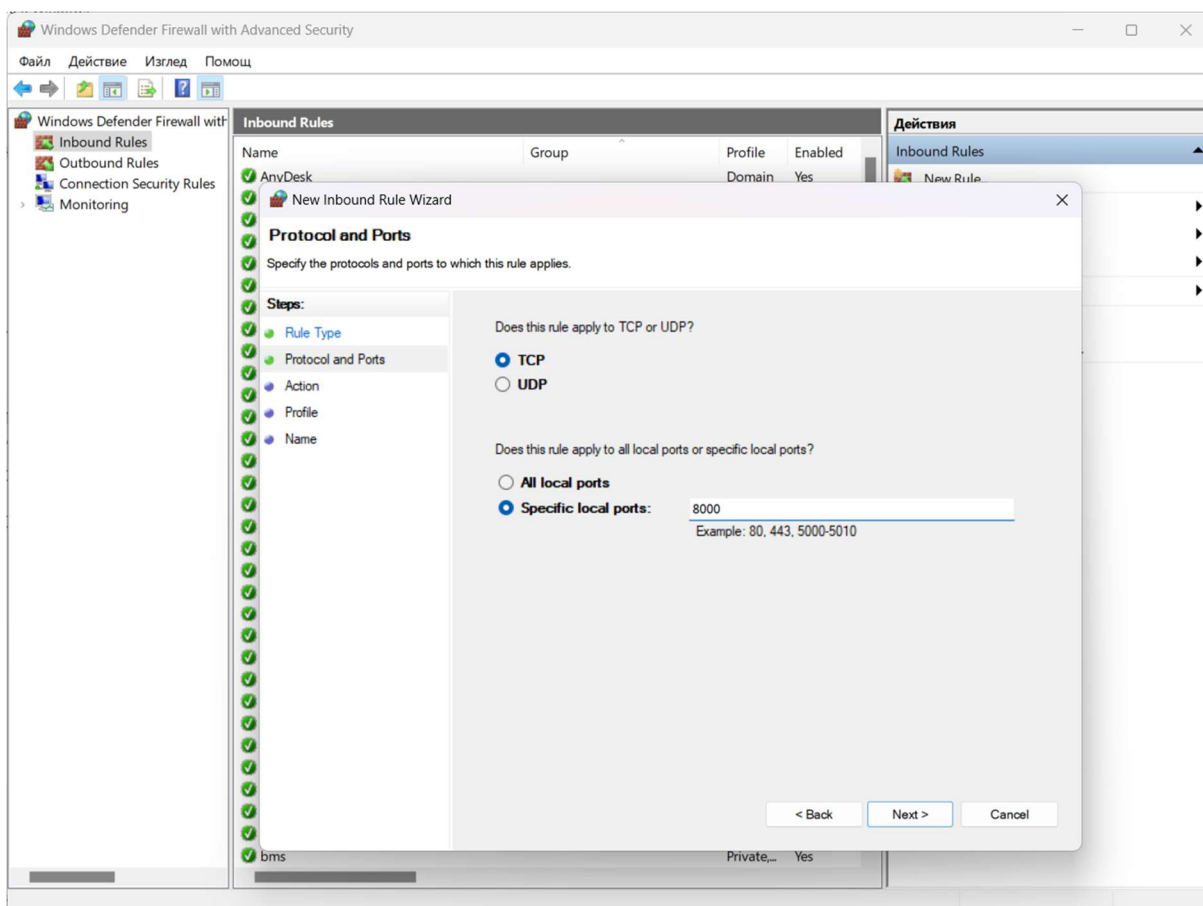
```
Connection-specific DNS Suffix . :  
Link-local IPv6 Address . . . . . : fe80::5cac:7ccd:351c:e1c3%27  
IPv4 Address. . . . . : 192.168.208.1  
Subnet Mask . . . . . : 255.255.240.0  
Default Gateway . . . . . :
```


Въз основа на този резултат, достъпът от мобилния браузър се осъществи чрез въвеждане на адреса <http://192.168.1.200:8000>.

5.2 ОТСТРАНЯВАНЕ НА МРЕЖОВИ ПРОБЛЕМИ (TROUBLESHOOTING)

По време на мрежовото тестване беше идентифициран стандартен за подобен тип архитектури проблем – първоначален отказ за свързване от страна на телефона (грешка "Site cannot be reached" или "Connection timed out").

Анализът на проблема показва, че причината не е в програмния код на Django, а в настройките за мрежова сигурност на операционната система (Windows Defender Firewall). По подразбиране защитната стена блокира входящите връзки към нестандартни портове като 8000.



Фигура 5. Конфигуриране на ново правило във Windows Defender Firewall за разрешаване на входящ HTTP трафик към порт 8000.

За отстраняването на този проблем бяха предприети следните системни административни стъпки:

1. Отваряне на разширените настройки на Windows Defender Firewall (Advanced Settings).
2. Създаване на ново правило за входящ трафик (Inbound Rule).
3. Разрешаване на TCP трафик през специфичен локален порт (в случая: 8000).
4. Прилагане на правилото за частни (Private) профили на мрежата.

След прилагането на тази настройка, комуникацията между телефона и лаптопа беше успешно възстановена и стабилизирана.

5.3 РЕЗУЛТАТИ ОТ ПРАКТИЧЕСКОТО ПРИЛОЖЕНИЕ

Резултатите от тестовете на системата са изключително позитивни и покриват изцяло заложените в техническото задание изисквания:

- Минимална латентност (Времезакъснение): Благодарение на локалния характер на мрежата (липса на рутване през външни интернет доставчици) и използването на асинхронни заявки с Axios, времето между натискането на бутона на телефона и смяната на слайда на екрана на лаптопа е практически незабележимо за човешкото око (под 50 милисекунди).
- Стабилност на интерфейса: Потребителският интерфейс, базиран на Vue.js и Bootstrap, се оразмерява перфектно на различни мобилни екрани (тествано на устройства с Android и iOS). Тъмната тема осигурява комфорт при работа в затъмнени презентационни зали.
- Визуална обратна връзка: Статус индикаторът (напр. "☒ Успешно: next") предоставя на презентатора сигурност, че командата е достигнала сървъра, което предотвратява многократното и излишно натискане на бутони при евентуално забавяне на системата.
- Безупречна интеграция с PowerPoint: Системата безпроблемно управлява всички основни функции на презентацията – стартиране на слайдшоу (F5), преминаване към следващ/предишен слайд (стрелки), спиране на презентацията (Esc), както и затъмняване на екрана в черно (B) или бяло (W) за временно привличане на вниманието на аудиторията към лектора.

5.4 АНАЛИЗ НА ИЗПЪЛНЕНИЕТО НА ЗАДАЧИТЕ

С оглед на поставените цели в Увода, може да се направи обоснован извод, че задачите са изпълнени в пълен обем:

- Изграден е стабилен Backend с Python и Django.
- Създадено е сигурно REST API чрез Django REST Framework.
- Реализирана е хардуерна емуляция чрез PyAutoGUI.
- Разработен е адаптивен мобилен Frontend с Vue.js.
- Системата не изисква инсталиране на допълнителни мобилни приложения – спазен е принципът за "Zero-install client".

5.5 ПРИНОСИ НА ДИПЛОМНИЯ ПРОЕКТ

Разработката на този дипломен проект носи няколко конкретни практически приноса:

- Създаване на функционален софтуерен продукт: Проектът не е просто теоретична концепция, а напълно работещо приложение, готово за реална експлоатация в образователна или бизнес среда.
- Икономическа ефективност: Системата предлага напълно безплатна алтернатива на скъпите хардуерни презентатори (clickers), като оползотворява вече наличния хардуер на потребителя (смартфон).
- Кросплатформена универсалност: Чрез избора на уеб-базиран клиентски интерфейс е преодоляно ограничението за зависимост от конкретна мобилна операционна система (работи еднакво добре на Safari, Chrome, Firefox Mobile).
- Разширяема архитектура: Проектът създава модулна софтуерна рамка (framework). Благодарение на ясното разделение между Frontend и Backend и използването на API, системата може лесно да бъде надградена в бъдеще (например чрез добавяне на нови бутони за управление на медийни плеъри като VLC, контрол на звука на операционната система или дори изключване на компютъра).

6. ЗАКЛЮЧЕНИЕ

Настоящият дипломен проект бе посветен на проектирането, разработването и внедряването на иновативна софтуерна система за дистанционно управление на десктоп приложения (с фокус върху Microsoft PowerPoint) чрез мобилно устройство. В хода на работата беше направен задълбочен анализ на съществуващите хардуерни и софтуерни решения за презентиране, като бяха идентифицирани техните основни недостатъци – необходимост от допълнителни финансови инвестиции, риск от хардуерна несъвместимост и неудобството от инсталиране на специализирани мобилни приложения.

За преодоляване на тези ограничения беше предложена и успешно реализирана уеб-базирана архитектура от тип "клиент-сървър", оперираща в рамките на локална безжична мрежа (Wi-Fi).

В процеса на разработка бяха изпълнени всички заложи в заданието задачи:

- Изграден е стабилен и надежден сървър (Backend) с помощта на езика Python и уеб рамката Django.
- Създаден е гъвкав приложно-програмен интерфейс (API) чрез Django REST Framework (DRF), който осигурява сигурна и бърза комуникация между устройствата.
- Интегрирана е библиотеката PyAutoGUI, чрез която мрежовите команди успешно се транслират във физически действия на ниво операционна система (емулация на клавиатура).
- Проектиран е интуитивен, адаптивен и лек потребителски интерфейс (Frontend) за мобилни устройства, използващ силата на Vue.js и Axios за асинхронно изпращане на команди без презареждане на страницата.
- Преодолени са мрежовите рестрикции на операционната система (конфигурация на Windows Defender Firewall), което гарантира безпроблемното свързване на смартфона със сървъра по локален IP адрес.

Постигнатите резултати доказват ефективността на избрания технологичен стек. Създадената система е напълно функционална, реагира с минимално времезакъснение (latency) и предоставя на презентатора пълна свобода на движение,

използвайки единствено собствения му смартфон (в духа на концепцията Bring Your Own Device).

Въпреки че текущата версия на системата изпълнява отлично основното си предназначение, нейната модулна архитектура предоставя широки възможности за бъдещо надграждане:

- Внедряване на автентикация: Добавяне на система за сигурност, при която достъпът до контролния панел на телефона изисква въвеждане на кратък PIN код, генериран от сървъра. Това ще предотврати неоторизиран достъп, ако в локалната мрежа има и други активни потребители.
- Управление на курсора на мишката: Използване на сензорния екран на телефона като тъчпад (Trackpad) за прецизен контрол над мишката.
- Разширяване на поддържаните приложения: Създаване на допълнителни интерфейси за управление на мултимедийни плейъри (напр. VLC, Spotify) или уеб браузъри, като контролният панел автоматично се адаптира според активното приложение на компютъра.
- Bluetooth алтернатива: Интегриране на резервен комуникационен канал през Bluetooth за ситуации, в които липсва надеждна Wi-Fi мрежа.

В заключение, дипломният проект демонстрира на практика как съвременните уеб технологии могат да бъдат комбинирани за създаването на достъпен, ефективен и лесен за използване софтуерен продукт, който решава реален проблем от ежедневието.

7. ЛИТЕРАТУРА:

1. Колисниченко, Денис, Адаптивен уеб дизайн с Bootstrap, Асеновци, С 2019
2. D.K Academy, Python – практическо програмиране. Асеновци, С 2022
3. Донълдсън, Тоби. Бързо ръководство:Програмиране с Python.АлексСофт, С 2017
4. Al Sweigart. Automate the Boring Stuff with Python: Practical Programming for Total Beginners. No Starch Press, 2nd Edition. (Основен източник за работата с модула PyAutoGUI и автоматизацията на GUI).
5. Axios Documentation. Официална техническа документация на HTTP клиента Axios. Достъпно на: <https://axios-http.com/docs/intro>
6. Bootstrap 5 Documentation. Ръководство за изграждане на адаптивен дизайн. Достъпно на: <https://getbootstrap.com/docs/5.0/getting-started/introduction/>
7. Django Documentation. Официална документация на уеб рамката Django (версия 4.x/5.x). Достъпно на: <https://docs.djangoproject.com/>
8. Django REST Framework. Ръководство за изграждане на Web APIs с Django. Достъпно на: <https://www.django-rest-framework.org/>
9. MDN Web Docs (Mozilla Developer Network). Справочник за уеб технологии: HTML, CSS, JavaScript и HTTP протоколи (CORS, CSRF защита). Достъпно на: <https://developer.mozilla.org/>
10. PyAutoGUI Documentation. Официално ръководство за използване на кросплатформения модул за контрол на мишка и клавиатура. Достъпно на: <https://pyautogui.readthedocs.io/>
11. Python Official Documentation. Справочник за стандартната библиотека на езика Python (версия 3.x). Достъпно на: <https://docs.python.org/3/>
12. Vue.js Documentation. Официално ръководство за прогресивната JavaScript рамка Vue (версия 3). Достъпно на: <https://vuejs.org/guide/introduction.html>
13. William S. Vincent. Django for APIs: Build web APIs with Python and Django. Welcome to Code, 2022. (Ръководство за правилното структуриране и маршрутизиране на API заявки).